AI-POWERED DATA CENTER RACK HEALTH MONITORING SYSTEM

23CSE365
Internet of Things

CB.SC.U4CSE23042 - NNN TEJESH
CB.SC.U4CSE23016 - D S VASISTA
CB.SC.U4CSE23223 - K GIRISH

MENTOR-
M SENTHIL KUMAR



AGENDA

- PROBLEM & MOTIVATION
- REQUIREMENT ANALYSIS
- SYSTEM ARCHITECTURE
- SENSOR & INTERFACE PLANNING
- PROTOTYPE DEMO
- TEAM CONTRIBUTION
- OUTCOMES & FUTURE SCOPE

REACTIVE ALERTS ARRIVE TOO LATE



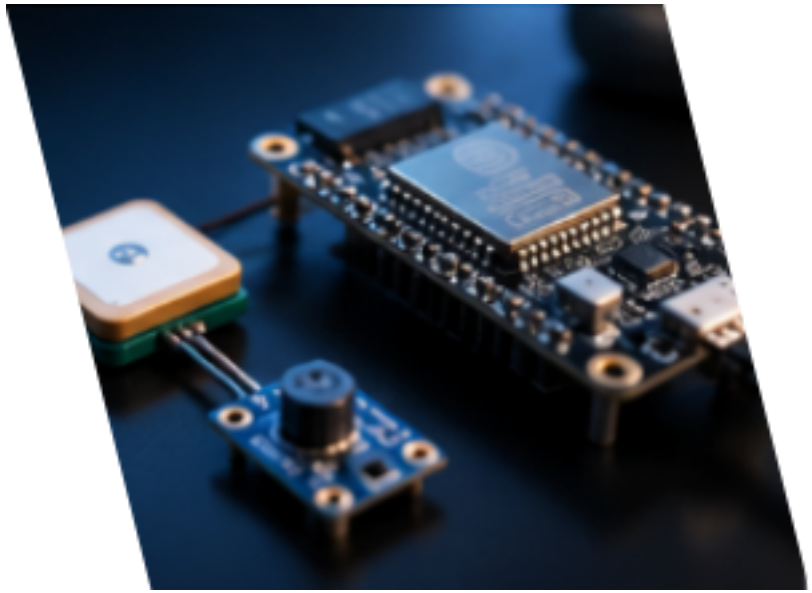
Rack failures build up gradually — from heat, humidity, smoke, vibration, or abnormal power draw

Most existing systems only alert after a threshold is already crossed, by when downtime has often begun

Small setups like college labs or edge servers rarely have any early-warning system at all — we're building one that predicts failure before it happens, not after

OBJECTIVES & MOTIVATION

- Monitor temperature, humidity, vibration, smoke, and power draw continuously
- Push this data to the cloud in real time
- Run a lightweight ML model to classify rack health
- Show live status on a dashboard and trigger alerts before failure



REQUIREMENT ANALYSIS: HARDWARE

Component	Purpose
ESP32	Main microcontroller, Wi-Fi enabled
DHT22	Temperature & humidity
INA219	Current & voltage (power draw)
MQ135 / MQ2	Air quality / smoke detection
SW420	Vibration sensing
OLED display	Local status readout
RGB LED + Buzzer	Local alert indication
Relay + 5V Fan	Simulated load / cooling response



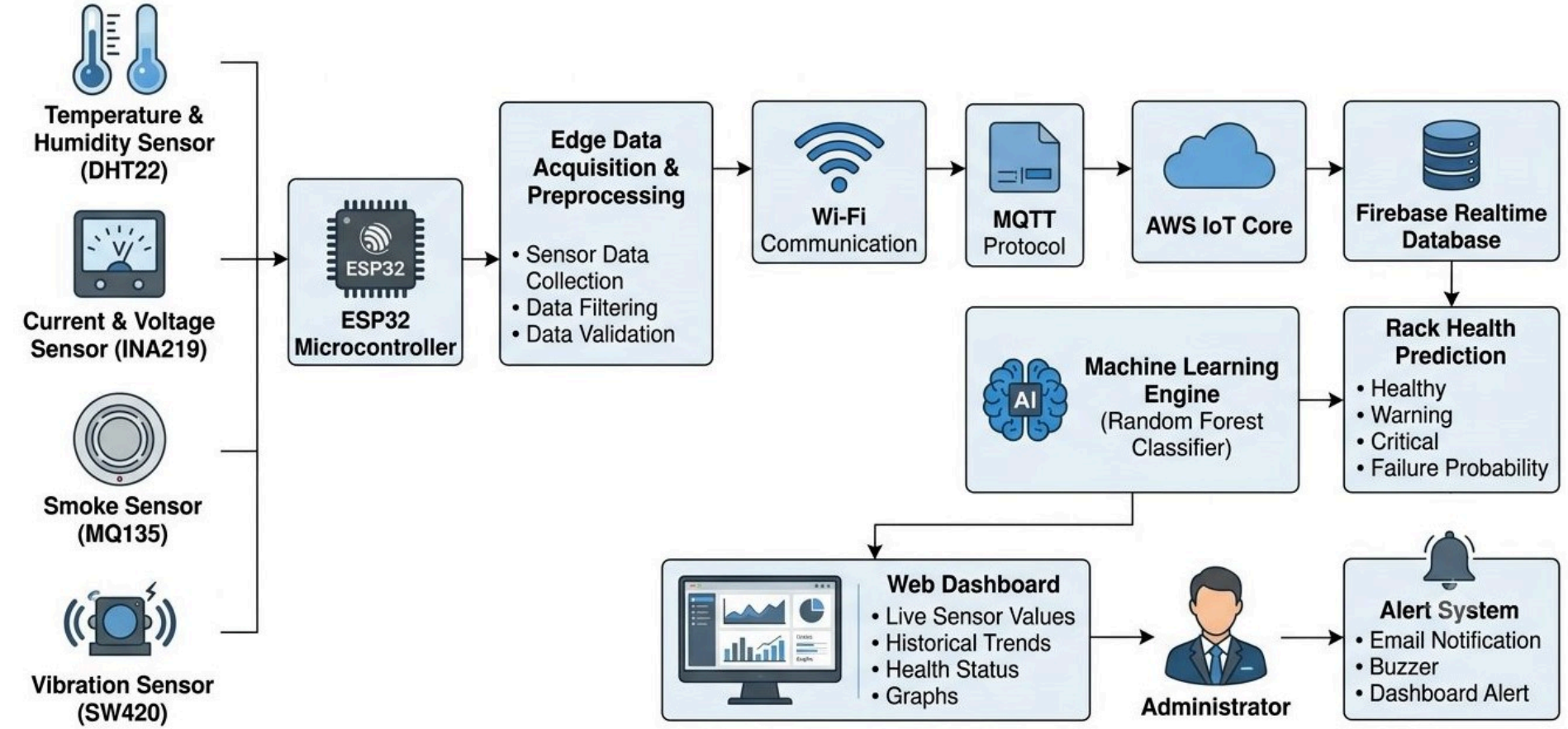
REQUIREMENT ANALYSIS: SOFTWARE, PROTOCOLS & CONSTRAINTS

- Firmware: Arduino IDE (C/C++)
- ML & backend: Python, Scikit-learn
- Messaging: MQTT
- Cloud: AWS IoT Core, Firebase
- Constraints we designed around: low-cost parts only, dependence on stable Wi-Fi, sensor-accuracy limits of low-cost modules

SUCCESS METRICS

- Prediction accuracy target: **> 90%**
- Alert latency: **< 3 seconds**
- Sensor update interval: **< 2 seconds**

SYSTEM ARCHITECTURE



WHY THIS ARCHITECTURE ?

ML inference runs in the **cloud**, not on the ESP32 — the microcontroller doesn't have the compute/memory for Scikit-learn models

ESP32's job is only sensing + transmission, keeping firmware simple and power-efficient

MQTT chosen over plain HTTP because it's lightweight, low-bandwidth, and built for many small, frequent messages — a better fit than HTTP's heavier per-request overhead

DATA FLOW


1 PHYSICAL LAYER LAYER
Mini Server Rack
Cooling Fans
DHT22 Temperature & Humidity Sensor
INA219
INA219 Current Sensor
MQ135 Smoke Sensor
SW420 Vibration Sensor
2 EDGE COMPUTING LAYER
ESP32
Local Data Acquisition
Sensor Fusion
Data Filtering
Edge Processing
3 COMMUNICATION LAYER
Wi-Fi
MQTT
MQTT Protocol
Secure Communication
4 CLOUD LAYER
AWS IoT Core
MQTT Broker
AWS Lambda
Firebase Realtime Database
5 AI ANALYTICS LAYER LAYE
Data Preprocessing
Feature Extraction
Random Forest Model
Failure Prediction
Rack Health Classification
6 APPLICATION LAYER LAYER
Web Dashboard
Live Graphs
Historical Trends
Alert Notification
Email / SMS

SENSOR & INTERFACE PLANNING

Sensor	Interface	Why this interface
DHT22	GPIO (single-wire)	Simple digital protocol, no bus needed for one sensor
INA219	I2C	Needs precise digital readback; I2C supports multiple devices on 2 wires
OLED	I2C	Shares the same bus as INA219 — saves pins
MQ135	ADC (analog)	Gas sensors output analog voltage proportional to concentration
SW420	GPIO (digital)	Vibration sensor gives simple threshold trigger, not analog range
Relay	GPIO	Simple on/off control signal

SIGNAL CONDITIONING PLAN

- DHT22 has a built-in digital conversion — no external conditioning needed
- MQ-series gas sensors need a warm-up period and a voltage-divider circuit before ADC reading is stable
- INA219 handles current/voltage sensing digitally, removing the need for manual op-amp scaling
- Basic software-side smoothing (moving average) applied before data is sent, to reduce sensor noise

ML MODEL

- Features used: temperature, humidity, current, voltage, smoke level, vibration
- Model: Random Forest Classifier
- Output classes: Healthy / Warning / Critical
- Currently trained on a synthetic dataset — planned to be replaced with real rack data once more field readings are collected



PROTOTYPE DEMONSTRATION

- Miniature rack simulated using three 5V fans as load
- Additional fans switched on to raise current draw artificially (simulating load spikes)
- INA219 tracks power draw, DHT22 tracks temperature, MQ135 tracks air quality, SW420 tracks vibration



INDIVIDUAL CONTRIBUTION

Member	Responsibility	What they actually did
N.Tejesh	Hardware	Sensor wiring, ESP32 firmware, circuit assembly
K.Girish	Cloud & Dashboard	AWS IoT Core setup, Firebase dashboard
D.Vasishta	Machine Learning	Dataset design, model training, feature selection
	Integration & Testing	End-to-end testing, documentation, this report

EXPECTED OUTCOMES, FUTURE SCOPE

Outcomes: reduced downtime, predictive (not reactive) maintenance, real-time visibility, architecture that scales to more racks

Future scope: move some inference to the edge (on-device ML), digital twin simulation, monitoring multiple racks at once, camera-based inspection




THANK YOU